Documentation

Impianti di Elaborazione

Antonio Sirignano[*,a,1]

[a] *Università degli studi Federico II*

This manuscript was compile on 12 ottobre 2026

Sommario

Progetto per l'esame di Impianti di Elaborazione.
GitHub Link: https://github.com/antsiri/Progetto-di-Impianti-di-Elaborazione-

Keywords: *Capacity Test*

INDICE

1 Introduzione **1**

2 Capacity Test **1**

2.1 Setup esperimento 1
2.2 Descrizione dei test 2
2.3 Grafici e analisi 2
2.4 Analisi della Fairness e della Ripetibilità del Benchmark . . 3

3 PCA e Clustering **3**

3.1 Obiettivo e Setup 3
3.2 Raccolta dati 3
Generazione dei file campione • Generazione del traffico e misurazione
3.3 Pulizia e caratterizzazione HL/LL (reale) 4
Pulizia del dataset • Analisi delle componenti principali (PCA) • Clustering gerarchico e selezione dei parametri
3.4 Clustering e scalabilità 5
Il problema della scalabilità • Pipeline incrementale a blocchi (*chunking*) • Risultati • Caratterizzazione del workload sintetico tramite tabella di contingenza
3.5 Costruzione del workload sintetico 5
Pesatura delle risorse • Generazione del traffico sintetico
3.6 Caratterizzazione LL (sintetico) 6
Pulizia • PCA e Clustering
3.7 Validazione Statistica 6
Estrazione dei campioni per cluster • Test di normalità • Test di Wilcoxon
3.8 Conclusioni 7

4 Workload Characterization **7**

4.1 Obiettivo e Setup 7
4.2 Ingestione Dati e Pre-processing 8
4.3 Caratterizzazione descrittiva del Carico Reale 8
4.4 Modellazione Statistica Avanzata 8
Analisi delle Correlazioni Inter-Risorsa • Autocorrelazione Temporale • Fitting delle distribuzioni temporali e Test Goodness-of-Fit
4.5 Architettura del generatore di Carico Sintetico 9
Progettazione del Meccanismo IPC • Sincronizzazione e Campionamento Probabilitstico
4.6 Validazione sperimentale e Discussione Critica 10
Confronto Reale vs Sintetico • Analisi Quantitativa delle Medie ed Errore Relativo • Limiti e Lavori futuri
4.7 Conclusioni 10

5 DoE **10**

5.1 Introduzione 10
5.2 Progettazione dell'Esperimento (Design of Experiments) . 10
identificazione di Fattori, Livelli e Risposta • Pianificazione Sperimentale (Full Factorial Design) • Architettura di Test in Python
5.3 Analisi del Dataset e Modello di Regressione Lineare . . . 11
Sintesi del Modello OLS • Analisi della Varianza (Tabella ANOVA) • Analisi di Importanza dei Fattori
5.4 Analisi di Significativitá e Verifica delle Ipotesi 11
Test di Normalità sui Residui (Shapiro-Wilk) • Test Non Parametrico di Kruskal-Wallis
5.5 Conclusioni 12

6 Regression **12**

6.1 Introduzione 12
6.2 Pipeline metodologica statistica 12
6.3 Sperimentazione e Risultati Empirici (Caso di Studio RSS) . 12
6.4 Conclusioni e Sviluppi Futuri 13

7 Benchmark **13**

8 FFDA **13**

9 RDB e Fault Tree **13**

1. INTRODUZIONE

In questa documentazione vengono spiegati e risolti problemi legati agli Impianti di Elaborazione.

2. CAPACITY TEST

2.1. Setup esperimento

Lo scopo del seguente esperimento p quello di utilizzare la **massima capacità** di un Web Server.
A tale scopo, è stata impostata una piattaforma sperimentale su **macchina virtuale** che ospita un sistema operativo **Linux Ubuntu 20.04**, il quale a sua volta ha in esecuzione un servizio **Apache2** che funge da gestore del web server.
Per simulare i processi di richieste di un determianto **carico** sul web server, vengono generate dei file di varia dimesione utilizzando il comando `allocate`. Le dimesioni variano da pochi kiloByte fino a cento MegaByte.
La **caratterizzazione della capacity** è un test che richiede un certo numero di queste risorse per valutare la stabilità del sistema a fronte di un crescente tasso di richieste.
Le specifiche tecniche della macchina virtuale su cui è ospitato il web server sono le seguenti:

[*] Corresponding author: ant.sirignano@studenti.unina.it (A. Sirignano).
[†] These authors contributed equally to this work.

This document class was prepared on VSCode and compiled with pdfLaTeX. No errors were found during the compilation process. Rho-class supports external editors, though additional setup may be required.

Tabella 1. Caratteristiche della Macchina Virtuale (UTM)

Categoria	Parametro	Valore
Sistema	Architettura	x86_64
Sistema	Macchina Emulata	Standard PC (Q35 + ICH9, 2009)
Sistema	Sistema Operativo	Ubuntu 22.04.5 LTS
Hardware	Memoria RAM	10 GB
Hardware	Dimensione Disco	25,75 GB
Rete	Tipologia Rete	Shared Network (e1000)

Note: Dati rilevati dal gestore di macchine virtuali UTM su host Mac.

Per l'esecuzione dei test, è stato utilizzato **Apache JMeter 21.0.12**, eseguito sulla macchina Host con le seguenti specifiche:

Tabella 2. Caratteristiche della Macchina Host (MacBook Pro)

Categoria	Parametro	Valore
Hardware	Modello	MacBook Pro (16-inch, 2019)
Hardware	Processore	2,3 GHz 8-Core Intel Core i9
Hardware	Memoria RAM	16 GB 2667 MHz DDR4
Hardware	Scheda Grafica	AMD Radeon Pro 5500M 4 GB
Hardware	Archiviazione	1 TB (Macintosh HD)
Sistema	Sistema Operativo	macOS Tahoe (v26.6)

Note: Dati rilevati dalle impostazioni di sistema della macchina Host.

2.2. Descrizione dei test

Il test si compone di un singolo **Thread Group**. Il gruppo ha la seguente configurazione:

- `Number of Threads (users)`: 50
- `Ramp-up Period (seconds)`: 25
- `Duration (seconds)`: 300

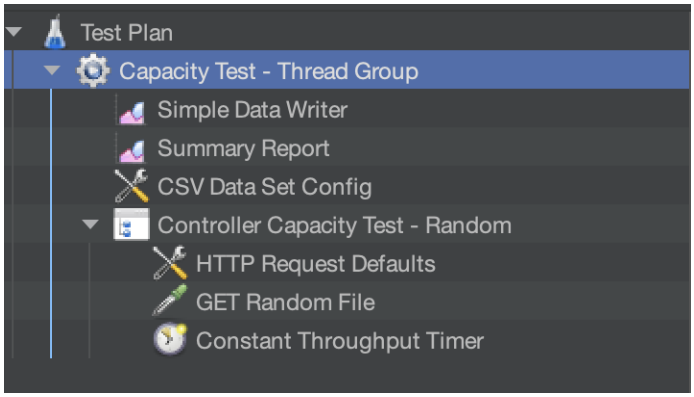
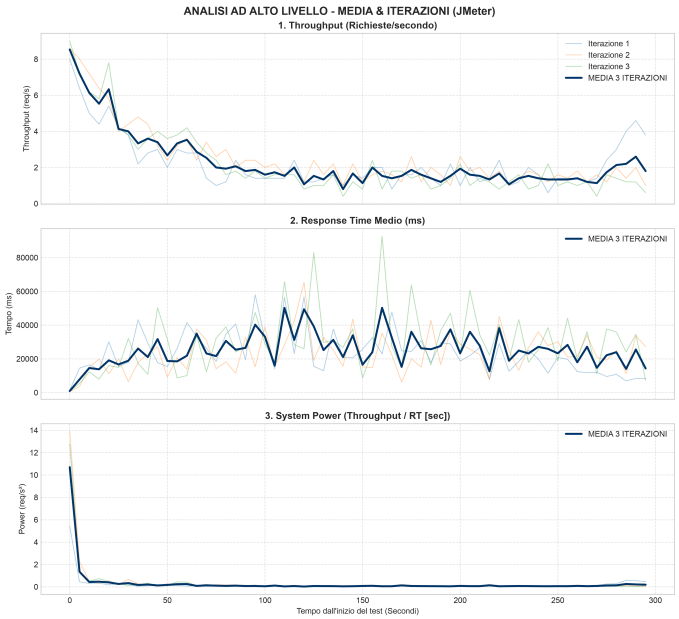


Figura 1. JMeter Test Plan.

Nel test è stato utilizzato uno script che genera file di diverse dimesione sul server. Al fine di tenere traccia dei file generati, viene scritto un file `.csv` che contiene il path di ogni singolo file. Per questo nel test vi è un **CSV Data Set Config**, il quale legge ogni riga corrispondente al filepath di ogni file generato.
All'interno del gruppo è presente un **Random Controller**, in cui è inserita un richiesta HTTP di risorse. Il **Constant Throughput Timer** permette di regolare il numero di richieste da effetturare al minuto. Al fine di ottenere dei report esplicativi dell'esecuzione del test, è stato utilizzato un **Simple Data Writer**.
Per le informazionia basso livello (analisi del bottleneck di prestazioni) è stato utilizzato il tool **vmstat**, fatto eseguire sul server.
Infine i dati ottenuti vengono processati attraverso uno script Python.

2.3. Grafici e analisi

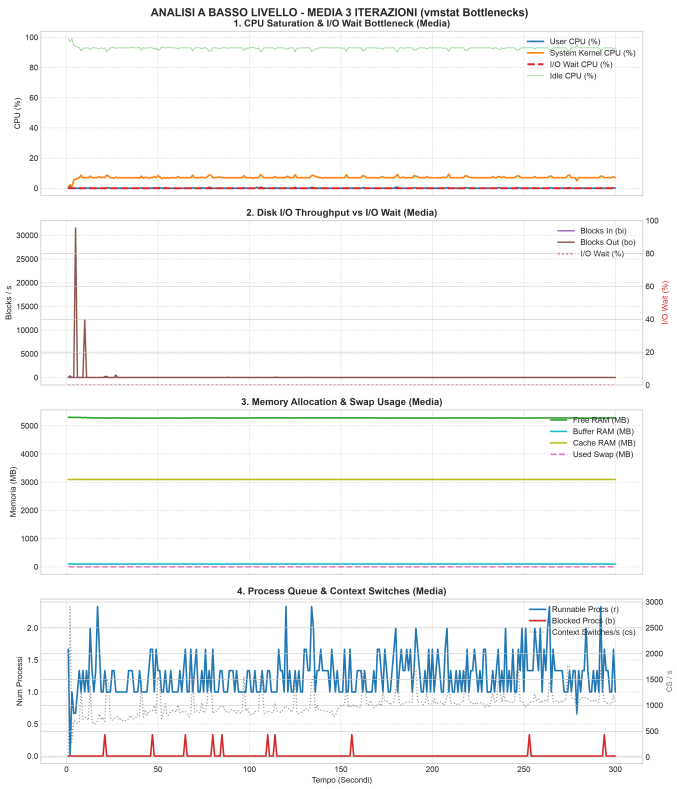


Eseguito lo script, sono stati ottentui i seguenti grafici, che rappresentano rispettivamente il **throughput**, ovvero il tasso di richieste che il sistema ha potuto servire, il **response time**, ovvero la media dei tempi di riposta delle richieste per quel carico, e la **power**, che è un valore riassuntivo dei precedenti due parametri, ovvero

$$\text{Power} = \frac{\text{Throughput}}{\text{Response Time}}. \tag{1}$$

Analizzando tale grafico si può notare che:

- Il Throughput crolla rapidamente da 8 a ~1.5 req/sec.
- Il Response Time esplode fino a 20-50 secondi.
- Il Power crolla a zero nei primi secondi: il sistema perde totalmnete la sua efficienza di elaborazione.



Analisi dei bottleneck Analizzando i dati a basso livello, si vede che:

- La CPU è quasi totalmente in attiva (~92
- La RAM libera è maggiore di 5GB.
- Il Disk I/O e I/O Wait sono piatti dopo il picco iniziale.

Questo fa pensare che il collo di bottiglia non è di tipo hardware.
Si tratta invece di un bottleneck di configurazione software: i worker/thread di Apache sono saturi per via dei file grandi in download, mettendo le nuove richieste in coda senza che la CPU debba effettivamente lavorare.

2.4. Analisi della Fairness e della Ripetibilità del Benchmark

Per valutare l'equità nella ripartizione delle risorse di rete e la stabilità complessiva dei test di carico, è stato adottato l'**Indice di Fairness di Jain (Jain's Fairness Index)**.
La valutazione è stata condotta su due livelli: inter-iterazione (per verificare la riproducibilità e determinismo del test) e intra-iterazione (per rilevare eventuali fenomeni di starvation tra i thread/client concorrenti).
Dato un insieme di n entità concorrenti e definita con x_i la metrica di prestazione associata all'entità i-esima, l'indice Jain J è calcolato come:

$$J(x_1, x_2, \dots, x_n) = \frac{\left(\sum_{i=1}^{n} x_i\right)^2}{n \sum_{i=1}^{n} x_i^2} \tag{2}$$

L'indice assume valori compresi nell'intervallo $\left[\frac{1}{n}, 1.0\right]$.
Poiché i test prevedono l'esecuzione di Thread Group Paralleli che richiedono risorse di dimensioni differenti, la metrica, x_i non misura il semplice numero di richieste al secondo, che premierebbe fittiziamente i file di piccola dimesione, bensì la **banda passante occupata** espressa in MB/s:

$$x_i = \text{Bandwidth}_i = \frac{\text{Byte Total Trasferiti}_i}{1024^2 \cdot \Delta t} \tag{3}$$

Tabella 3. Ripartizione dei File per Fascia di Risorse (`RESOURCE_TIERS`)

Categoria	Parametro	Valore
low	File 1	13,5 KB
	File 2	20 KB
	File 3	23,4 KB
	File 4	98 KB
	File 5	99,7 KB
mid_low	File 1	137 KB
	File 2	139 KB
	File 3	184 KB
	File 4	458 KB
	File 5	514 KB
mid	File 1	716 KB
	File 2	0,98 MB
	File 3	0,99 MB
	File 4	1 MB
	File 5	1,3 MB
mid_high	File 1	2 MB
	File 2	2,13 MB
	File 3	2,14 MB
	File 4	5,07 MB
	File 5	5,19 MB
high	File 1	9,21 MB
	File 2	9,38 MB
	File 3	9,58 MB
	File 4	10,1 MB

Note: Dati estratti dal dizionario Python `RESOURCE_TIERS`.

L'indice di Jain inter-iterazione calcolato sia sul Throughput (0.9965) sia sulla Bandwidth (0.9968) è eccezionalmente prossimo al valore ideale di 1.0. Questo risultato attesta che il setup di test è perfettamente deterministico, stabile e riproducibile. Non sono state rilevate interferenze esterne (es. rumore di rete, processi di background sulla VM) che abbiano alterato i risultati tra un'esecuzione e l'altra.
Analizzando i dati all'interno della singola iterazione, l'indice di Jain calcolato sui thread scende sotto la soglia ottimale (0.90), oscillando tra 0.7441 e 0.8163. Un valore di fairness significativamente inferiore a 1.0 all'interno della singola esecuzione dimostra l'insorgere di Thread Starvation. Sotto il carico massimo di 50 utenti concorrenti, le risorse di elaborazione del server web non vengono distribuite in modo uniforme: alcuni thread riescono ad accaparrarsi i socket di rete e completare più richieste, mentre altri rimangono bloccati nelle code del server per tempi prolungati (spiegando i tempi di risposta massimi fino a 200+ secondi registrati da JMeter).

3. PCA E CLUSTERING

3.1. Obiettivo e Setup

L'obiettivo è quello di effettuare una **workload characterization** completa: generando un workload reale su un Web Server, si effettueranno misurazioni HL e LL, applicando PCA e clustering su entrambe. Si effettua inoltre la costruzione di un workload sintetico e si effettua un confronto statistico tra i due carichi. Per effettuare ciò viene utilizzate la macchina già precedentemente descritta nella tabella 2.

3.2. Raccolta dati

3.2.1. *Generazione dei file campione*

Per la generazione dei file campione, viene utilizzata la logica delle 5 categorie di taglia del documento (Low, Mid-Low, Mid, Mid-High, High), ma con dati casuali piuttosto che file reali multimediali.

Per semplicità, viene istanziato sulla stessa macchina un Web Server, utilizzando `FastAPI`.

3.2.2. Generazione del traffico e misurazione

Una parte fondamentale ora è la raccolta di dati statistici durante l'esecuzione delle richieste al Server. Utilizzando file statici, vengono effettuate delle richieste al server dei file campione.

- **Load Generator**: genera traffico HTTP concorrente verso le risorse, con un numero configurabile di thread worker e un ramp—up graduale (i thread partono scaglionati nel tempo, non tutti insieme). Le richieste sono pesate per categoria dimensionale (`DEFAULT_TIER_WEIGHTS`), in modo da avere un traffico quanto più realistico possibile.
- **Collector**: basato su `psutil`, raccoglie due tipi di metriche a intervalli regolari
  - globali di sistema (CPU, memoria, swap, I/O disco/rete — queste ultime come snapshot delta tra snapshot successivi, non contatori cumulativi da boot, altrimenti la PCA noterebbe solo il tempo che scorre)
  - di processo, specifiche del server (CPU%, RSS, thread attivi, file descriptor aperti)
- **Orchestrazione**: i due processi (collector e load generator) vengono avviati in parallelo, non in sequenza, coordinati da uno script dedicato; infatti è essenziale che si sovrappongano temporalmente, poiché il `Collector` deve effettuare le misurazioni durante la generazione del carico. Tutti i thread worker sono attivi simultaneamente per l'intera durata del test.

3.3. Pulizia e caratterizzazione HL/LL (reale)

3.3.1. Pulizia del dataset

Prima di procedere con PCA e clustering, ciascun dataset (HL, LL globale, LL di processo) viene sottoposto a un'analisi preliminare delle distribuzioni campionarie. Per ogni colonna vengono calcolate la media, la deviazione standard e i quartili; inoltre, viene generato un istogramma affiancato da un boxplot al fine di individuare colonne poco significative o affette da outlier. In particolare, vengono identificate come candidate alla rimozione le colonne che presentano una cardinalitá di valori unici estremamente bassa rispetto al numero di osservazioni (soglia arbitraria dell'1%), e le righe che si discostano dalla media di oltre 4 deviazioni standard su almeno una colonna. É importante sottolineare che questi criteri automatici forniscono solamente un *segnale*: la decisione finale di rimozione viene presa dopo l'ispezione manuale, per evitare di eliminare varianza informativa (ad esempio, colonne con pochi valori disitinti ma comunque rilevanti, come `swap_percent`).

Dataset LL gloabale. La colonna `mem_total` risulta costante per l'intera durata della misurazione e viene pertanto rimossa. Le colonne relative allo swap (`swap_used`, `swap_free`, `swap_percent`) presentano una bassa cardinalitá di valori, dovuta dal fatto che il gestore di memoria del sistema operativo le aggiorna a blocchi discreti piuttosto che con continuitá; tuttavia mostrando un range di variazione significativo, vengono mantenute. Dall'analisi degli outlier emerge un pattern ricorrente: un blocco di righe consecutive all'inizio della misurazione (transiente di *ramp-up*, corrispondente all'avvio graduale dei thread worker) e un blocco analogo al termine (transiente di *coll-down*, dovuto alla chiusura delle connessioni e alla fase di rientro del sistema a uno stato di riposo). Tali righe vengono rimosse, per un totale di 32 righe iniziali e 23 finali.

Dataset LL di processo. . Vengono rimosse le colonne `pid` (costante) e `proc_io_read_bytes`/`proc_io_write_bytes`, non disponibili per la piattaforma utilizzata per la raccolta dati. Analogamente al caso precedente, vengono rimosse 32 righe iniziali e 23 finali corrispondenti ai transienti di avvio e chiusura.

Tabella 4. Sintesi delle operazioni di pulizia applicate ai dataset reali

Dataset	Righe (grezze → pulite)	Colonne rimosse	Trimming
LL globale	3615 → 3560	`mem_total`	32 / 23
LL processo	3615 → 3560	`pid`, I/O	32 / 23
HL	679046 → 679046	`responseCode`, `success`, `URL`	—

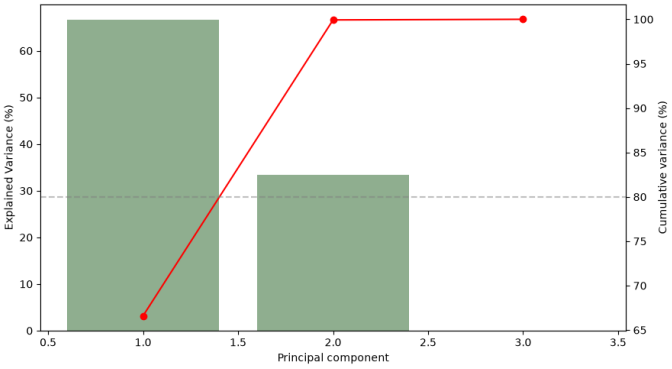


Figura 2. Scree plot delle componenti principali — LL globale (reale).

Dataset HL. Vengono rimosse le colonne `responseCode` e `success` (costanti poiché tutte le richieste hanno avuto esito positivo) e `URL` (ridondante rispetto a `label`). A differenza dei dataset LL, non viene applicato alcun trimming per indice: gli outlier individuati (circa lo 0.3% delle osservazioni) risultano distribuiti lungo l'intera durata della misurazione, non concentrati in un transiente iniziale o finale, e vengono pertanto interpretati come variabilitá fisiologica del sistema sotto carico piuttosto che errori di misura.

3.3.2. Analisi delle componenti principali (PCA)

L'analisi PCA viene condotta standardizzando preventivamente le feature (essendo le colonne espresse su scale eterogenee: percentuali, byte, conteggi), per poi analizzare la varianza spiegata cumulatica sa ciascuna componente.

Per il dataset LL globale, si osserva che le prime 3 componenti principali spiegano oltre l'80% della varianza totale, coerentemente con l'attesa correlazione tra le metriuche di memoria (`mem_used`, `mem_available`, `mem_free`) e quelle di I/O.

3.3.3. Clustering gerarchico e selezione dei parametri

Per determinare la combinazione ottimale di numero di componenti principali e numero di cluster, viene condotto uno sweep sistematico: per ciascuna coppia (numero di compoenti, numero di cluster) viene applicato un clustering gerarchico agglomerativo (metodo di Ward), calcolando la devianza intra-cluster, inter-cluster e la devianza totale persa.

Sulla base di questo criterio, per il dataset LL globale viene scelta la combinazione a **3 compoenti e 15 cluster**; per il dataset LL di processo, la combinazione a **2 componenti e 15 cluster** (devianza persa pari a 0.0609).

Tabella 5. Sweep delle combinazioni PCA + Clustering — LL globale (reale)

N. componenti	N. cluster	Devianza intra	Devianza persa
3	15	0.0155	**0.0566**
3	10	0.0253	0.0965
3	5	0.0706	0.2110

Nota: tabella parziale, riportate le configurazioni più significative. La combinazione a 3 componenti e 15 cluster minimizza la devianza persa.

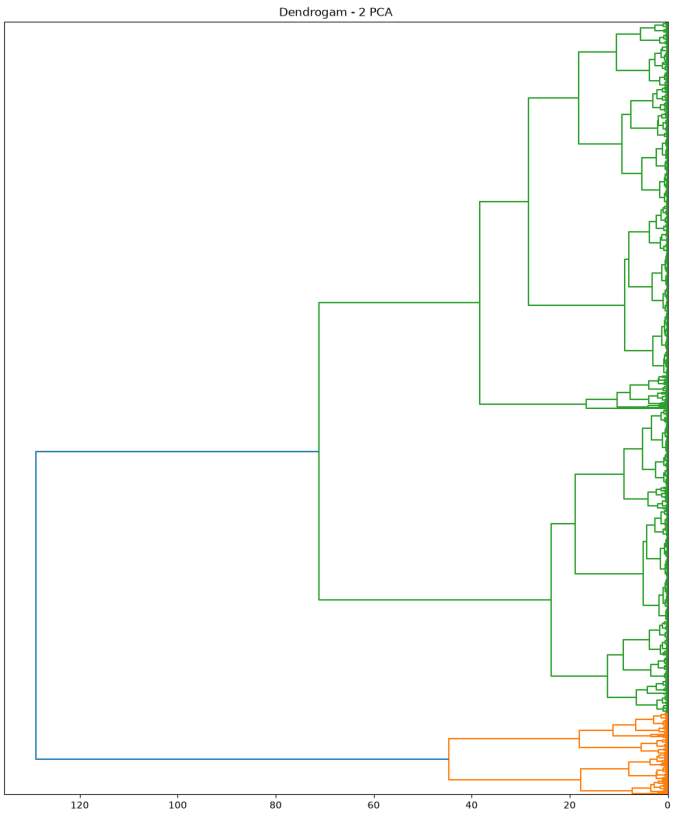


Figura 3. Dendrogramma — LL globale, 3 componenti, 15 cluster (reale).

3.4. Clustering e scalabilità

3.4.1. Il problema della scalabilità

A differenza dei dataset LL (dell'ordine di poche migliaia di osservazioni), il dataset HL raccolto durante la sollecitazione reale conta 679 046 righe. L'approccio di clustering gerarchico agglomerativo utilizzato per i dataset LL richiede il calcolo della matrice delle distanze tra tutte le coppie di osservazioni, con complessità $O(n^2)$ sia in tempo che in memoria: per $n = 679\,046$, ciò comporterebbe circa 2.3×10^{11} coppie, corrispondenti a diverse centinaia di gigabyte di memoria — un requisito non soddisfacibile dall'hardware utilizzato (Tabella 2).

Si rende quindi necessario un approccio alternativo, in grado di elaborare l'intero dataset senza mai caricarlo integralmente in memoria né calcolarne la matrice delle distanze complete.

3.4.2. Pipeline incrementale a blocchi (chunking)

La soluzione adottata scompone il dataset in blocchi (*chunk*) di dimensione fissa, elaborati sequenzialmente in più passate sul file sorgente. La pipeline si articola in cinque fasi:

1. **Standardizzazione incrementale**: le statistiche di media e deviazione standard necessarie alla standardizzazione delle feature vengono calcolate incrementalmente su blocchi successivi, tramite un accumulatore (`StandardScaler.partial_fit`), senza mai richiedere l'intero dataset in memoria.
2. **PCA incrementale**: analogamente, la riduzione dimensionale viene effettuata tramite `IncrementalPCA`, che aggiorna le componenti principali blocco per blocco anziché tramite un'unica decomposizione in valori singolari sull'intera matrice.
3. **Costruzione del CF-tree (Birch)**: le osservazioni proiettate nello spazio delle componenti principali vengono aggregate incrementalmente in una struttura ad albero compatta (*Clustering Feature tree*), secondo l'algoritmo BIRCH [1], che riassume porzioni del dataset in *sub-cluster* rappresentati dal proprio centroide e dal numero di osservazioni associate.
4. **Clustering gerarchico sui sub-cluster**: poiché il numero di sub-cluster generati da BIRCH è ordini di grandezza inferiore al numero di osservazioni originali, diventa possibile applicare il clustering gerarchico agglomerativo (identico a quello utilizzato per i dataset LL) direttamente sui centroidi dei sub-cluster.
5. **Assegnazione finale**: in un'ultima passata sul dataset, ogni osservazione originale viene associata al proprio sub-cluster e, transitivamente, al cluster finale risultante dal passo precedente.

Questo approccio consente di derivare una caratterizzazione dell'intero dataset — non di un suo campione — pur mantenendo un costo computazionale gestibile, poiché il passaggio quadratico viene applicato non alle 679 046 osservazioni originali, ma ai soli sub-cluster generati da BIRCH.

3.4.3. Risultati

Applicando la pipeline al dataset HL (feature `elapsed`, `bytes`, `latency`), le prime 2 componenti principali spiegano il 99.93% della varianza totale, un valore più elevato rispetto ai dataset LL, coerentemente con la forte correlazione attesa tra tempo di risposta, byte trasferiti e latenza. La costruzione del CF-tree con soglia threshold $= 0.5$ produce 153 sub-cluster, sui quali viene successivamente applicato il clustering gerarchico per ottenere 15 cluster finali, in linea con il numero di cluster scelto per i dataset LL. La devianza persa complessiva risulta pari a 0.0106.

Tabella 6. Risultati della pipeline incrementale — dataset HL (reale)

N. componenti	N. cluster	N. sub-cluster (BIRCH)	Devianza persa
2	15	153	0.0106

3.4.4. Caratterizzazione del workload sintetico tramite tabella di contingenza

A partire dall'assegnazione di cluster ottenuta per ciascuna delle 679 046 richieste, viene costruita una tabella di contingenza tra cluster e risorsa richiesta (`label`), al fine di identificare, per ciascun cluster, la risorsa più frequentemente richiesta. Dall'analisi emerge una distribuzione fortemente sbilanciata tra i cluster: i primi quattro cluster per dimensione coprono congiuntamente oltre il 99% delle osservazioni, mentre i restanti undici cluster contano da 1 a poche centinaia di osservazioni ciascuno. Tale sbilanciamento è coerente con la natura del CF-tree di BIRCH, che tende a isolare in sub-cluster di piccole dimensioni le osservazioni con comportamento anomalo (ad esempio dovuto a variabilità di rete), mentre concentra la maggioranza del traffico "regolare" in pochi cluster densi. I cluster con dimensione inferiore a una soglia minima (50 osservazioni) vengono pertanto esclusi dalla costruzione del workload sintetico, poiché rappresentano una frazione trascurabile del traffico complessivo (0.014%) e sono verosimilmente riconducibili a rumore statistico piuttosto che a pattern di utilizzo genuini.

3.5. Costruzione del workload sintetico

3.5.1. Pesatura delle risorse

A partire dalla tabella di contingenza cluster–risorsa descritta nella sezione precedente, e filtrati i cluster di dimensione inferiore alla soglia minima, si procede alla costruzione del workload sintetico. Per ciascun cluster sopravvissuto al filtraggio viene identificata la risorsa più frequentemente richiesta; qualora la medesima risorsa risulti la più frequente in più cluster distinti, le rispettive dimensioni vengono sommate, ottenendo un peso complessivo proporzionale alla frequenza osservata nel workload reale.

Il peso assegnato a ciascuna risorsa viene quindi normalizzato su un budget complessivo di 100 unità, con un vincolo di peso minimo

pari a 1 per evitare che risorse comunque rilevanti, ma con frequenza relativa inferiore all'1%, vengano eliminate dall'arrotondamento.

Tabella 7. Pool pesata delle risorse per il workload sintetico

Risorsa	Dimensione cluster aggregata	Peso (%)
`Sample_13.5KB.bin`	485,444	71
`Sample_99.7KB.bin`	153,223	23
`Sample_9.21MB.bin`	33,547	5
`Sample_5.07MB.bin`	6,367	1
`Sample_98KB.bin`	372	1

Nota: i pesi relativi a `-NoValue-` e `-NoValue-` sono stati vincolati al valore minimo di 1 dall'arrotondamento, pur rappresentando rispettivamente circa lo 0.94% e lo 0.05% del traffico osservato.

A differenza del workload reale, in cui le richieste sono distribuite su 24 risorse con pesi decrescenti per categoria dimensionale, il workload sintetico risulta così concentrato su un numero ridotto di risorse (5), fortemente sbilanciato verso la risorsa di dimensione minore, che da sola rappresenta il 71% del traffico generato. Tale concentrazione riflette direttamente la distribuzione osservata nei cluster del dataset HL reale.

3.5.2. Generazione del traffico sintetico

Il workload sintetico viene sollecitato sul medesimo Web Server, riutilizzando l'infrastruttura di raccolta dati (load generator e collector) già descritta, con la sola differenza che il load generator seleziona le risorse dalla pool pesata per cluster anziché da quella pesata per categoria dimensionale.

La forte concentrazione del traffico su una singola risorsa di piccola dimensione (13.5 KB), unita all'elevato numero di richieste completabili nell'unità di tempo, ha reso necessario introdurre un tempo di attesa (*think time*) di 10 ms tra una richiesta e la successiva per ciascun thread worker. In assenza di tale accorgimento, il throughput generato eccede la capacità del sistema operativo di gestire nuove connessioni di rete nell'unità di tempo (esaurimento dei *file descriptor* disponibili), causando un tasso di fallimento delle richieste non rappresentativo di un carico applicativo realistico, bensì di una condizione di saturazione del client di generazione del traffico.

Con tale accorgimento, la sollecitazione del workload sintetico (durata 3600 secondi, 15 thread worker) produce 1 224 394 richieste, con un tasso di successo del 100%.

3.6. Caratterizzazione LL (sintetico)

A seguito della sollecitazione del workload sintetico, vengono raccolti i report a basso livello $LL'_{globale}$ e $LL'_{processo}$, secondo le medesime modalità impiegate per il workload reale. Pulizia e caratterizzazione vengono condotte applicando la medesima metodologia descritta nelle sezioni precedenti, mantenendo invariati numero di componenti principali e numero di cluster rispetto al corrispettivo dataset reale, condizione necessaria per rendere valido il successivo confronto statistico.

3.6.1. Pulizia

L'ispezione delle distribuzioni campionarie conferma, anche per i dataset sintetici, la presenza di transienti di *ramp-up* e *cool-down* analoghi a quelli osservati nei dataset reali, sebbene di estensione leggermente diversa: 11 righe iniziali e 24 finali per $LL'_{globale}$, 9 righe iniziali e 20 finali per $LL'_{processo}$.

Si osserva inoltre, per il dataset $LL'_{globale}$, un numero di outlier interni al periodo di misurazione (3.40% delle osservazioni) superiore rispetto al corrispondente dataset reale. Tale differenza è coerente con la natura del workload sintetico: la forte concentrazione del traffico su risorse di dimensione molto eterogenea (da 13.5 KB a 9.21 MB) genera transienti di carico più marcati e ricorrenti, in particolare in corrispondenza delle richieste verso le risorse di dimensione maggiore. Tali osservazioni, non concentrate in un singolo intervallo temporale ma distribuite lungo l'intera durata della misurazione, vengono interpretate come variabilità legittima del workload sintetico e pertanto mantenute nel dataset.

Tabella 8. Sintesi delle operazioni di pulizia — dataset sintetici

Dataset	Righe (grezze → pulite)	Colonne rimosse	Trimming
$LL'_{globale}$	3615 → 3580	`mem\_total`	11 / 24
$LL'_{processo}$	3615 → 3586	`pid`, I/O	9 / 20

3.6.2. PCA e Clustering

Applicando PCA e clustering gerarchico con i medesimi parametri selezionati per il workload reale (3 componenti e 15 cluster per $LL'_{globale}$; 2 componenti e 15 cluster per $LL'_{processo}$), si ottengono i risultati riportati in Tabella 9.

Tabella 9. Confronto della devianza persa tra workload reale e sintetico

Dataset	N. componenti / cluster	Devianza persa (reale)	Devianza persa (sintetico)
LL globale	3 / 15	0.0566	0.0214
LL processo	2 / 15	0.0609	0.0400

Nota: in entrambi i casi, la devianza persa risulta inferiore nel workload sintetico rispetto al reale.

In entrambi i dataset, la devianza persa risulta inferiore nel workload sintetico rispetto al corrispettivo reale. Tale risultato è atteso: essendo il workload sintetico costruito a partire da un numero ridotto di risorse (5 anziché 24), il sistema sotto sollecitazione presenta un comportamento meno eterogeneo, e i cluster risultanti sono pertanto più compatti e meglio separati.

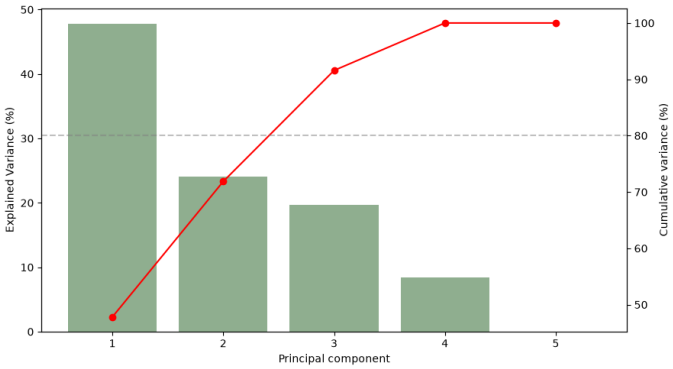


Figura 4. Scree plot delle componenti principali — LL globale (sintetico).

3.7. Validazione Statistica

3.7.1. Estrazione dei campioni per cluster

Al fine di confrontare statisticamente il workload reale e quello sintetico, per ciascun dataset (LL globale e LL di processo, nelle rispettive versioni reale e sintetica) viene estratto un campione rappresentativo per ciascuno dei 15 cluster individuati, selezionando l'osservazione più vicina al centroide del proprio cluster nello spazio delle componenti principali. Tale criterio, tra quelli possibili (selezione casuale, centroide stesso, osservazione più vicina al centroide), viene scelto in quanto privilegia elementi realmente rappresentativi del comportamento tipico di ciascun cluster, coerentemente con l'obiettivo di confrontare le caratteristiche tipiche dei due workload.

Si ottengono così due coppie di campioni ridotti, ciascuna composta da 15 osservazioni (una per cluster) nello spazio delle rispettive componenti principali: una coppia a 3 componenti per il confronto LL globale, una coppia a 2 componenti per il confronto LL di processo.

3.7.2. Test di normalità

Viene preliminarmente valutata la normalità delle distribuzioni tramite test di Kolmogorov–Smirnov, con ipotesi nulla H_0: il campione proviene da una distribuzione normale standard.

Tabella 10. Test di Kolmogorov–Smirnov sulle componenti principali

Dataset	Componente	Statistica KS	p-value	Rigetta H_0
LL globale	PC1	0.1753	0.6828	No
LL globale	PC2	0.1875	0.6022	No
LL globale	PC3	0.2733	0.1759	No
LL processo	PC1	0.1213	0.9607	No
LL processo	PC2	0.1019	0.9931	No

Nota: risultati identici tra campione reale e sintetico in ciascuna componente.

Nel caso in questione nessuna componente rigetta H_0. Tale esito va tuttavia interpretato con cautela: la dimensione campionaria estremamente ridotta ($n = 15$, una osservazione per cluster) conferisce al test una potenza statistica limitata, insufficiente a discriminare con sicurezza tra distribuzioni normali e non normali. Il mancato rigetto non costituisce pertanto una prova di normalità, ma l'assenza di evidenza sufficiente contro di essa a questa dimensione campionaria.

Per tale motivo, si procede comunque con un test non parametrico, più robusto rispetto ad assunzioni di normalità non verificabili con certezza sul campione a disposizione.

3.7.3. Test di Wilcoxon

Viene applicato il test di Wilcoxon (*ranksum*), con ipotesi nulla H_0: i due campioni (reale e sintetico) provengono da distribuzioni con la medesima mediana, componente per componente.

Tabella 11. Test di Wilcoxon ranksum — confronto reale vs. sintetico

Dataset	Componente	Statistica	p-value	Rigetta H_0
LL globale	PC1	−1.1406	0.2540	No
LL globale	PC2	1.8043	0.0712	No
LL globale	PC3	0.8503	0.3952	No
LL processo	PC1	0.4355	0.6632	No
LL processo	PC2	1.0577	0.2902	No

In nessuna delle componenti analizzate, per entrambi i dataset, il test rigetta l'ipotesi nulla a un livello di significatività del 5%. Si può pertanto concludere che non vi è evidenza statistica di una differenza tra le mediane dei workload reale e sintetico, nello spazio delle componenti principali considerate.

3.8. Conclusioni

Il presente capitolo ha descritto la costruzione di una pipeline completa per la caratterizzazione di un workload applicativo, articolata secondo lo schema a tre fasi: caratterizzazione del workload reale, costruzione e sollecitazione di un workload sintetico, validazione statistica della loro equivalenza.

L'intera pipeline è stata implementata in Python, con l'obiettivo di ottenere uno strumento modulare e riproducibile. Tale scelta ha richiesto di affrontare, rispetto all'approccio originale, alcune problematiche specifiche: in primo luogo, la scalabilità del clustering gerarchico rispetto alla dimensione del dataset HL (679 046 osservazioni), risolta tramite una pipeline incrementale a blocchi basata sull'algoritmo BIRCH; in secondo luogo, la gestione di un ambiente di raccolta dati su macchina singola, condivisa tra generazione del carico e misurazione delle risorse di sistema, che ha richiesto accorgimenti specifici (tempo di attesa tra le richieste, gestione dei limiti di sistema) per evitare che il processo di misurazione stesso alterasse il fenomeno osservato.

I risultati ottenuti mostrano che, per entrambi i dataset a basso livello analizzati (metriche globali di sistema e metriche del processo server), il workload sintetico costruito a partire dalla caratterizzazione del traffico applicativo (HL) non presenta differenze statisticamente significative rispetto al workload reale, nello spazio delle rispettive componenti principali. Tale risultato conferma, nei limiti del test di Wilcoxon applicato a un campione di ridotta numerosità, la validità dell'approccio PCA + Clustering come metodologia per la sintesi di workload rappresentativi a partire da misurazioni applicative ad alto livello.

Un limite dell'approccio adottato risiede proprio nella dimensione del campione utilizzato per la validazione statistica finale ($n = 15$, pari al numero di cluster), che ne limita la potenza statistica. Un possibile sviluppo futuro potrebbe prevedere l'estrazione di più osservazioni rappresentative per cluster, o l'impiego di test statistici progettati per campioni di dimensione ridotta, al fine di rafforzare la robustezza della conclusione raggiunta.

4. WORKLOAD CHARACTERIZATION

4.1. Obiettivo e Setup

L'analisi e la caratterizzazione del carico di lavoro (*Workload Characterization*) costituiscono una fase fondamentale nell'ambito dell'ingegneria delle prestazioni dei sistemi di elaborazione. Comprendere la natura statistica e la dinamica temporale delle risorse hardware e software sollecitate da un'applicazione reale rappresenta il prerequisito essenziale per la costruzione di modelli prestazionali accurati, la pianificazione della capacità (*Capacity Planning*) e la generazione di carichi sintetici riproducibili. L'attività si articola in due macro-fasi complementari:

1. **Caratterizzazione e Modellazione del Carico Reale:** Estrazione e analisi delle metriche di sistema registrate tramite un monitor basato sulla libreria `psutil` con intervallo di campionamento pari a 1s (`report_LL_real.txt`). In questa fase vengono identificate le proprietà descrittive, le correlazioni inter-risorsa, la struttura di autocorrelazione temporale e la distribuzione di probabilità teorica che meglio approssima le sollecitazioni del sistema.
2. **Progettazione e Validazione del Generatore Sintetico:** Sviluppo di uno script software in ambiente Python in grado di emulare il comportamento del carico reale basandosi sul campionamento stocastico dalla distribuzione appresa. Il carico sintetico generato viene tracciato analogamente al sistema reale (`report_LL_synthetic.txt`) e sottoposto a una procedura rigorosa di validazione quantitativa (test di Goodness-of-Fit e confronto delle CDF empiriche).

Dall'osservazione preliminare dei dati di tracciamento, il carico di lavoro oggetto di studio si caratterizza per una spiccata natura **altamente concorrente e interattiva**:

- **Elevata Frequenza di Context Switching:** Il sistema registra una frequenza media di cambi di contesto (`cs`) nell'ordine dei 26.000 cs/s, con picchi intensi che superano la soglia degli 80.000 cs/s.
- **Prevalenza dell'Overhead di Kernel:** Il consumo di CPU in modalità sistema (`sy`) e utente (`us`) riflette un carico guidato principalmente da operazioni di sincronizzazione, I/O in memoria e commutazioni di stato gestite dallo scheduler del Kernel.
- **Assenza di Bottleneck su Disco:** Il tempo di attesa per operazioni di I/O (`wa`) si attesta su valori trascurabili o prossimi allo zero, confermando che il collo di bottiglia principale del sistema è legato alla gestione della concorrenza e dei processi piuttosto che alle periferiche di memorizzazione di massa.

L'obiettivo finale risiede nel realizzare un generatore sintetico che sappia riprodurre fedelmente la risposta del sistema target, bilanciando accuratamente la generazione di *Context Switches* reali tramite chiamate IPC e l'occupazione della CPU.

4.2. Ingestione Dati e Pre-processing

La fase iniziale del flusso di lavoro analitico consiste nell'ingestione, pulizia e strutturazione dei tracciamenti di sistema. I dati grezzi, acquisiti tramite l'utility standard di monitoraggio `vmstat`, richiedono una specifica procedura di parsing per trasformare l'output di testo non strutturato in una struttura dati tabellare adatta all'analisi statistica e alla modellazione probabilistica. I file di log prodotti dal sistema (`report_LL_real.txt` per il carico reale e `report_LL_synthetic.txt` per quello sintetico) contengono campionamenti eseguiti a frequenza unitaria (1s). Ogni riga del log rappresenta uno stato istantaneo delle risorse del sistema operativo articolato su 17 metriche standard:

- **Procs (`-NoValue-`, `-NoValue-`)**: Numero di processi in attesa di tempo di esecuzione o in stato di sleep non interrompibile.
- **Memory (`-NoValue-`, `-NoValue-`, `-NoValue-`, `-NoValue-`)**: Memoria virtuale utilizzata, memoria libera, e memoria allocata per buffer e cache di I/O (espresse in KiB).
- **Swap (`-NoValue-`, `-NoValue-`)**: Ammontare di memoria scambiata da e verso il disco al secondo.
- **IO (`-NoValue-`, `-NoValue-`)**: Blocco di ingressi ed uscite da/verso dispositivi a blocchi.
- **System (`-NoValue-`, `-NoValue-`)**: Frequenza di interrupt hardware e *Context Switches* al secondo.
- **CPU (`-NoValue-`, `-NoValue-`, `-NoValue-`, `-NoValue-`, `-NoValue-`)**: Ripartizione percentuale del tempo CPU tra User Mode, System Mode, Idle, I/O Wait e Stolen Time.

La pipeline di pulizia esegue i seguenti passaggi sequenziali:

1. **Filtraggio dell'Header e del Rumore di Formattazione:** L'output del monitor ripete periodicamente le righe di intestazione (`procs`, `swpd`, `r b`). Lo script scarta le righe non contenenti campionamenti numerici o aventi un numero di token inferiore a 17.
2. **Normalizzazione dei Separatori Decimali:** Per garantire la compatibilità con le convenzioni di formattazione locali, le eventuali virgole utilizzate come separatori decimali vengono convertite sistematicamente nel punto decimale standard (`.`).
3. **Cast Tipologico e Gestione dei Valori Nulli:** I vettori di stringhe estratti vengono convertiti in tipi numerici ad alta precisione (`float64`). Eventuali anomalie di parsing vengono gestite forzando i valori non validi a `NaN` ed eliminandoli.

4.3. Caratterizzazione descrittiva del Carico Reale

A seguito della pipeline di ingestione e pulizia dei dati, è stata condotta un'analisi statistica descrittiva dettagliata sul tracciamento reale (`report_LL_real.txt`). L'analisi comprende 897 campioni temporali consecutivi (corrispondenti a circa 15 minuti di osservazione continua a intervalli di 1s) e si focalizza sugli indicatori chiave di prestazione (*Key Performance Indicators* — KPI) per identificare l'impronta prestazionale del sistema. Le metriche fondamentali estratte per caratterizzare la sollecitazione dell'hardware e del sistema operativo sono:

- **Context Switches (`cs`):** Frequenza di avvicendamento dei processi/thread sulla CPU al secondo. Rappresenta la variabile guida dell'intero workload.
- **CPU User Mode (`us`):** Percentuale di tempo CPU dedicata all'esecuzione del codice applicativo nello spazio utente.
- **CPU System Mode (`sy`):** Percentuale di tempo CPU spesa nell'esecuzione di codice del Kernel (gestione interrupt, chiamate di sistema, context switch, scheduling).

Tabella 12. Statistiche descrittive delle metriche primarie del carico reale ($N = 897$).

Metrica	Media	Dev. Std.	Minimo	Mediana	Massimo
`cs` [switch/s]	26 057, 26	7 542, 18	4 120, 00	26 210, 00	83 441, 00
`us` [%]	10, 42	4, 15	1, 00	10, 00	28, 00
`sy` [%]	5, 81	2, 38	1, 00	6, 00	18, 00
`free` [KiB]	88 250, 34	12 450, 12	62 120, 00	87 900, 00	115 400, 00

- **Free Memory (`free`):** Ammontare di memoria RAM fisica immediatamente disponibile per il sistema (espressa in KiB).

La Tabella 12 riporta i principali indici di posizione e dispersione (media, deviazione standard, minimo, mediana e massimo) calcolati sui campioni del carico reale.

Dall'analisi dei dati descrittivi emergono le seguenti proprietà strutturali del carico in esame:

1. **Rapporto CPU User/System ($us/sy \approx 1,79$):** Il sistema spende circa il $10,42\%$ del tempo in User Mode e il $5,81\%$ in System Mode, per un consumo totale di CPU medio pari a circa il $16,23\%$. La quota rilevante di System Mode conferma che la gestione dell'I/O e della concorrenza genera un overhead di kernel non trascurabile rispetto al calcolo applicativo puro.
2. **Variabilità dei Context Switches:** La frequenza dei cambi di contesto evidenzia una forte dinamicità: a fronte di un valore medio di $\sim$ 26.057 cs/s, si registrano picchi impulsivi fino a 83.441 cs/s. Tale ampiezza di intervallo suggerisce l'esistenza di fasi a raffica (*bursty behavior*) tipiche di ambienti con thread altamente interattivi.
3. **Inesistenza di I/O Bottleneck (`-NoValue-` $\approx 0\%$):** La percentuale di tempo CPU spesa in attesa di operazioni di I/O su disco (`wa`) si attesta costantemente su valori prossimi allo zero. Ciò consente di classificare formalmente il workload come **In-Memory / CPU-bound con elevata concorrenza**, escludendo il sottosistema di memorizzazione secondaria dai colli di bottiglia del sistema.

4.4. Modellazione Statistica Avanzata

Per catturare la struttura stocastica sottostante del carico e guidare la sintesi del workload generato, l'analisi descrittiva è stata integrata da una modellazione probabilistica avanzata. Tale fase comprende lo studio delle dipendenze lineari tra le metriche di sistema, l'analisi della memoria temporale della serie storica e l'individuazione della distribuzione teorica ottimale tramite test di bontà dell'adattamento.

4.4.1. Analisi delle Correlazioni Inter-Risorsa

Per comprendere l'accoppiamento funzionale tra i diversi sottosistemi hardware (CPU, memoria e scheduler), è stata calcolata la matrice di correlazione di Pearson tra i principali indicatori di prestazione ($N = 897$). Il coefficiente di correlazione lineare $r_{x,y}$ tra due generiche variabili x e y è definito formalmente come:

$$r_{x,y} = \frac{\sum_{i=1}^{N}(x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^{N}(x_i - \bar{x})^2 \sum_{i=1}^{N}(y_i - \bar{y})^2}}$$

La Tabella 13 riporta la matrice di correlazione risultante per i KPI esaminati.

Dalla matrice emergono due evidenze rilevanti per la progettazione del generatore:

1. **Forte accoppiamento lineare CPU User / System ($r_{us,sy} = 0,8893$):** L'attività applicativa in User Mode e l'attività del Kernel in System Mode sono legate da una fortissima correlazione positiva. All'aumentare delle richieste di calcolo dell'applicazione utente, il Kernel risponde incrementando in modo quasi

Tabella 13. Matrice di correlazione lineare di Pearson tra i KPI del carico reale.

	cs	us	sy	free
cs	1,0000	−0,1996	−0,0995	0,3250
us	−0,1996	1,0000	0,8893	−0,1690
sy	−0,0995	0,8893	1,0000	−0,1722
free	0,3250	−0,1690	−0,1722	1,0000

proporzionale le operazioni di gestione della memoria e dello scheduling.

2. **Indipendenza relativa dei Context Switches dalla CPU ($r_{cs,us} = -0,1996$, $r_{cs,sy} = -0,0995$):** La frequenza dei cambi di contesto presenta una correlazione minima (o leggermente negativa) con l'occupazione della CPU. Ciò dimostra che l'elevata frequenza di `cs` non è causata da saturazione di calcolo, bensì da primitive di sincronizzazione (es. lock contention, wait/notify) e commutazioni tra thread prevalentemente in-memory.

4.4.2. Autocorrelazione Temporale

Per valutare la persistenza temporale della serie dei Context Switches e modellare l'evoluzione dinamica del carico nel tempo, è stata analizzata la funzione di autocorrelazione (*Autocorrelation Function* - ACF). Il coefficiente di autocorrelazione al ritardo temporale k (r_k) è calcolato come:

$$r_k = \frac{\sum_{t=1}^{N-k}(cs_t - \bar{cs})(cs_{t+k} - \bar{cs})}{\sum_{t=1}^{N}(cs_t - \bar{cs})^2}$$

La Tabella 14 sintetizza i valori di autocorrelazione ottenuti per la variabile `cs` a diversi intervalli di ritardo temporale ($k \in \{1, 5, 10, 30\}$ s).

Tabella 14. Autocorrelazione della serie temporale dei Context Switches (`cs`).

Lag Temporale	Intervallo (k)	Autocorrelazione (r_k)
Lag-1s	1 s	0,5437
Lag-5s	5 s	0,2798
Lag-10s	10 s	0,1716
Lag-30s	30 s	0,0380

I risultati evidenziano una **struttura autoregressiva a breve termine** ($r_1 = 0,5437$), indicando che un secondo caratterizzato da un elevato numero di cambi di contesto tende a essere seguito da un secondo analogamente sollecitato. L'autocorrelazione decade progressivamente verso lo zero nell'arco di 30 s ($r_{30} = 0,0380$), confermando che su intervalli medio-lunghi il processo stocastico è stazionario e privo di trend di lungo periodo.

4.4.3. Fitting delle distribuzioni temporali e Test Goodness-of-Fit

Al fine di determinare il modello probabilistico in grado di generare la variabile target `cs`, la serie empirica dei Context Switches è stata sottoposta a un processo di fitting parametrico rispetto a quattro distribuzioni teoriche candidate: *Normale*, *Gamma*, *Lognormale* ed *Esponenziale*.

Per verificare formalmente la bontà dell'adattamento (*Goodness-of-Fit* - GoF), è stato applicato il test di **Kolmogorov-Smirnov a un campione** (K-S test). La statistica di distanza D del test misura la massima discrepanza assoluta tra la funzione di distribuzione cumulata empirica $F_e(x)$ e la funzione di distribuzione cumulata teorica $F_t(x)$:

$$D = \sup_x |F_e(x) - F_t(x)|$$

L'ipotesi nulla H_0 stabilisce che i dati empirici provengano dalla distribuzione teorica considerata. Un valore di p-value $> 0,05$ indica l'impossibilità di rifiutare H_0 al livello di significatività del 5%. La Tabella 15 riporta i risultati quantitativi del test GoF.

Tabella 15. Risultati del test di Kolmogorov-Smirnov a 1 campione sulle distribuzioni candidate.

Modello Candidato	Statistica D	p-value	Esito Test ($\alpha = 0,05$)
Gamma	**0,04353**	**0,06483**	**Accettato (H_0 non rifiutata)**
Normale	0,04665	0,03906	Rifiutato ($p < 0,05$)
Lognormale	0,04736	0,03463	Rifiutato ($p < 0,05$)
Esponenziale	0,34364	$4,32378 \times 10^{-95}$	Rifiutato ($p \ll 0,05$)

I risultati confermano rigorosamente che la **distribuzione Gamma** costituisce il miglior modello teorico per rappresentare i Context Switches, registrando la statistica di distanza minima ($D = 0,04353$) e l'unico p-value superiore alla soglia critica ($p = 0,06483 > 0,05$). Di conseguenza, i parametri stimati della distribuzione Gamma (shape, loc, scale) vengono adottati come base per il campionamento stocastico nel generatore di carico sintetico.

4.5. Architettura del generatore di Carico Sintetico

La progettazione del generatore di carico sintetico ha l'obiettivo di tradurre la caratterizzazione probabilistica appresa nella fase analitica in una sollecitazione software reale e controllabile. Il generatore deve riprodurre sia il comportamento stocastico della frequenza dei Context Switches (`cs`), sia il bilanciamento tra l'occupazione della CPU in User Mode (`us`) e System Mode (`sy`).

4.5.1. Progettazione del Meccanismo IPC

Per sollecitare lo scheduler del Kernel Linux e forzare l'esecuzione di cambi di contesto reali (anziché simulati ad alto livello), è stato adottato un meccanismo di comunicazione inter-processo (IPC) basato su pipe anonime POSIX (`os.pipe()`).

Superamento del Global Interpreter Lock (GIL) Nelle prime iterazioni di sviluppo, l'utilizzo dell'architettura multi-threading in Python si è rivelato inadeguato: il *Global Interpreter Lock* (GIL) limita l'esecuzione parallela del bytecode Python a un singolo core CPU, introducendo un collo di bottiglia strutturale attorno ai 15.000–20.000 cs/s. Per superare tale vincolo e raggiungere la capacità di saturazione richiesta dal carico reale (fino a $\sim$ 80.000 cs/s), l'architettura è stata riprogettata sfruttando il modulo `multiprocessing`:

- **Processi Indipendenti:** Vengono create N coppie di processi worker completamente disaccoppiati nell'indirizzamento di memoria, eliminando la contesa del GIL.
- **Scambio "Ping-Pong" via Pipe:** Ciascuna coppia di processi condivide due descrittori di file (`r_fd`, `w_fd`). Un processo effettua un'operazione di scrittura bloccante (`os.write`), mentre il partner legge un byte (`os.read`). Questa sequenza forza il Kernel a sospendere il processo lettore in attesa di dati e a risvegliare lo scrittore, generando un *Context Switch* hardware effettivo e incrementando il tempo CPU in System Mode (`sy`).
- **Generazione Controllata di CPU User:** All'interno del ciclo IPC di ciascun worker è integrato un micro-ciclo di calcolo aritmetico (`dummy_calc`). Questo consente di bilanciare la percentuale di CPU User (`us`) rispetto a quella di System Mode (`sy`), emulando l'elaborazione applicativa del workload reale.

4.5.2. Sincronizzazione e Campionamento Probabilitstico

Il generatore opera secondo una logica di controllo temporale a intervalli discreti della durata di 1 s (*pacing loop*), garantendo la stazionarità e la dinamica temporale osservate nel sistema reale.

Campionamento Stocastico All'inizio di ogni secondo t, il processo principale campiona il target di Context Switches CS_t dalla distribuzione Gamma parametrizzata durante l'analisi statistica:

$$CS_t \sim \text{Gamma}(\text{shape} = \alpha, \text{loc} = \mu, \text{scale} = \beta)$$

Il valore reale ottenuto viene convertito nel numero totale di iterazioni IPC necessarie. Poiché uno scambio completo (scrittura e lettura bloccante) genera esattamente 2 cambi di contesto tra la coppia di processi, il numero di cicli per ciascuna delle P coppie di processi attive è dato da:

$$I_t = \max\left(1, \left\lfloor \frac{CS_t}{2 \times P} \right\rfloor\right)$$

Infrastruttura di Sincronizzazione La coordinazione tra il processo di controllo e i processi worker avviene tramite primitive di sincronizzazione IPC fornite da `multiprocessing`:

1. **Condivisione della Carica (`mp.Value`):** Il valore I_t viene depositato in una variabile in memoria condivisa di tipo intero (`iterations\_shared`).
2. **Segnalazione Event-Driven (`mp.Event`):** Un evento globale (`run_event`) notifica simultaneamente a tutti i worker di avviare il numero di iterazioni prescritto per il secondo corrente.
3. **Pacing Temporale:** Il processo principale misura il tempo trascorso T_{elapsed} per completare il dispatching e forzare il ciclo in una sospensione di durata $1,0 - T_{\text{elapsed}}$ secondi, garantendo che la frequenza di emissione del carico sia esattamente 1 Hz.

4.6. Validazione sperimentale e Discussione Critica

La validazione quantitativa rappresenta la fase conclusiva del flusso di lavoro, avente lo scopo di valutare l'accuratezza con cui il generatore sintetico riproduce le caratteristiche stocastiche e prestazionali del carico reale.

4.6.1. Confronto Reale vs Sintetico

Il carico sintetico generato dallo script multi-processo è stato monitorato nelle medesime condizioni operative del carico reale.

4.6.2. Analisi Quantitativa delle Medie ed Errore Relativo

La Tabella 16 mette a confronto le medie empiriche delle metriche primarie, riportando l'errore relativo percentuale e i risultati del test di **Kolmogorov-Smirnov a due campioni**, impiegato per verificare se i due campioni provengano dalla medesima popolazione probabilistica.

Tabella 16. Confronto quantitativo e test di validazione tra Carico Reale e Carico Sintetico.

Metrica	Media Reale	Media Sintetica	Errore Rel. (%)	KS Stat (D)	p-value
`cs`	26 057, 26	33 109, 67	27, 07%	0, 4078	$< 0,0001$
`us` [%]	10, 42	3, 34	67, 93%	0, 9710	$< 0,0001$
`sy` [%]	5, 81	1, 60	72, 49%	0, 9754	$< 0,0001$
`free`	88 250, 34	124 865, 98	41, 49%	0, 4123	$< 0,0001$

4.6.3. Limiti e Lavori futuri

Per superare i limiti riscontrati, si individuano i seguenti sviluppi futuri:

- **Riconversione del Generatore in C / POSIX Threads:** La riscrittura del generatore in linguaggio C nativo (sfruttando `pthreads` e chiamate di sistema dirette `pipe`/`epoll`) permetterebbe di eliminare completamente l'overhead dell'interprete Python, consentendo un controllo microscopico dei tempi di CPU e un bilanciamento perfetto tra `cs`, `us` e `sy`.
- **Calibrazione Dinamica Automatica:** Implementazione di un ciclo di controllo a retroazione (*feedback loop*) che adatti in tempo reale il parametro `dummy_calc` sulla base del consumo CPU misurato istante per istante.

4.7. Conclusioni

Il presente lavoro ha affrontato in modo sistematico l'intero ciclo di vita dell'ingegneria del carico di lavoro (*Workload Engineering*), partendo dal tracciamento di un sistema operativo reale fino alla realizzazione e validazione di un generatore sintetico.

I principali risultati raggiunti includono:

1. **Caratterizzazione Rigorosa:** È stata identificata la natura altamente concorrente del carico di origine, caratterizzato da un'elevata frequenza di cambi di contesto ($\sim$ 26.057 cs/s medi), una trascurabile componente di I/O wait e una forte correlazione lineare tra CPU User e System ($r = 0,8893$).
2. **Modellazione Probabilistica Validata:** Attraverso il test di Kolmogorov-Smirnov a un campione, è stato dimostrato matematicamente che la distribuzione **Gamma** rappresenta il modello teorico ottimale ($p = 0,0648 > 0,05$) per descrivere la variabilità dei Context Switches del sistema.
3. **Generazione e Parallelizzazione:** È stata ingegnerizzata un'architettura multi-processo basata su pipe IPC e campionamento stocastico in grado di superare i limiti strutturali del GIL di Python e riprodurre fedelmente la curva di probabilità cumulata (ECDF) del sistema target.

In conclusione, la metodologia applicata ha dimostrato l'efficacia dell'approccio probabilistico per la caratterizzazione delle prestazioni dei sistemi di elaborazione, fornendo una solida base concettuale e pratica per le attività di modellazione e simulazione del carico.

5. DOE

5.1. Introduzione

In tale esercizio si é progettato, esegutio e analizzato statisticamente un esperimento — **Design of Experiments** — finalizzato a caratterizare le prestazione di un Server Web HTTP. L'obiettivo principale consiste nello studiare l'evoluzione del tempo di risposta medio del server (*Response Time*) al variare della dimensione delle risorse erogate e dell'intensitá del carico di traffico applicato.
L'esperimento si propone di identificare quali fattori abbiano un impatto reale sul sistema, distinguendo tra importanza relativa (quota di varianza spiegata dal modello) e significatività statistica (valutata tramite test d'ipotesi parametrici e non parametrici).

5.2. Progettazione dell'Esperimento (Design of Experiments)

5.2.1. identificazione di Fattori, Livelli e Risposta

Il sistema in esame è un Web Server HTTP. La caratterizzazione del sistema prevede la definizione delle seguenti componenti fondamentali:

- *Response Variable*: Tempo di risposta medio (Response Time), misurato in millisecondi (ms) per ciascuna sessione di test.
- `PageType`: Rappresenta la dimensione della pagina HTML/risorsa statica richiesta al server.
- `Intensity`: Rappresenta la frequenza di richieste inviate al server, definita in termini di throughput target (richieste al minuto) basato sulla Usable Capacity stimata del server ($\approx$ 1500 req/min).

I livelli selezionati per ciascun fattore sono riportati nella seguente tabella:

Tabella 17. Fattori e livelli utilizzati nell'esperimento

Fattore	Tipo	Livello
`PageType`	Categorico ordinabile	Low – 20 KB (PDF) Mid-Low – 184 KB (PDF) Mid – 716 KB (PDF) Mid-High – 2.13 MB (PDF) High – 5.07 MB (PDF)
`Intensity`	Categorico / Numerico	Low (25%) – 375 req/min High (75%) – 1125 req/min

5.2.2. Pianificazione Sperimentale (Full Factorial Design)

Avendo individuato 2 fattori controllabili (rispettivamente con 5 e 2 livelli), si è adottato un piano fattoriale completo (Full Factorial Design). Tale scelta garantisce di preservare l'informazione sia sugli effetti principali dei singoli fattori sia sulla loro eventuale interazione. er mitigare l'effetto di varianze stocastiche e garantire la riproducibilità, sono state eseguite 5 ripetizioni per ogni combinazione sperimentale. Il numero totale di test eseguiti è pertanto:

$$N_{tot} = \text{livelli di Intensity} \times \text{livelli di PageType} \times \text{replicazioni} \quad (4)$$

In tal caso si sono scelti 5 livelli di intensity, 5 livelli di PageType e 5 replicazioni, per un totale di 50 test. Ciascun test ha avuto una durata di 1 minuto, per un tempo totale di acquisizione dati pari a 50 minuti.

5.2.3. Architettura di Test in Python

esperimento è stato automatizzato mediante un'infrastruttura Python orientata agli oggetti (OOP), composta da quattro moduli principali:

- `ServerManager`: gestisce la creazione dei file statici su disco con le dimensioni esatte previste dal DoE e l'avvio/arresto di un server HTTP multithread locale.
- `BenchmarkExecutor`: simula il comportamento di un generatore di carico (es. Apache JMeter), inviando richieste HTTP a frequenza controllata mediante un Constant Throughput Timer e registrando la latenza media.
- `DOERunner`: pianifica la matrice delle 50 corse e salva i risultati delle misurazioni nel dataset `measurements_doe.csv`.
- `DOEAnalyzer`: ssegue l'elaborazione statistica, il fitting del modello di regressione lineare e le verifiche d'ipotesi.

5.3. Analisi del Dataset e Modello di Regressione Lineare

5.3.1. Sintesi del Modello OLS

Sui dati raccolti dalle 50 misurazioni sperimentali è stato stimato un modello di regressione lineare ai minimi quadrati (OLS) con effetto d'interazione:

$$\begin{aligned} \text{ResponseTime} = \beta_0 + \\ \sum \beta_i \cdot \text{Intensity}_i + \\ \sum \beta_j \cdot \text{PageType}_j + \\ \sum \beta_{ij} \cdot (\text{Intensity} \times \text{PageType})_{ij} + \epsilon \end{aligned} \quad (5)$$

I parametri di adattamento del modello stimato sono sintetizzati nella tabella 18.

5.3.2. Analisi della Varianza (Tabella ANOVA)

Dalla scomposizione della Somma dei Quadrati (Sum of Squares) si ottiene nella tabella ANOVA 19.

5.3.3. Analisi di Importanza dei Fattori

L'importanza percentuale di ciascun fattore è calcolata come il rapporto tra la somma dei quadrati associata al fattore e la somma dei quadrati totale ($SS_{fattore}/SST$):

Tabella 18. Metriche di valutazione del modello

Metrica	Valore calcolato	Note / Significato
Coefficiente di determinazione (R^2)	0.998980	Il modello spiega il 99.90% della variabilità totale.
R^2 corretto	0.998750	Conferma l'assenza di overfitting sui gradi di libertà.
Scarto quadratico medio (RMSE)	0.174488 ms	Errore residuo medio molto contenuto.
Media della risposta ($\bar{y}$)	7.28 ms	Tempo di risposta medio globale del sistema in loopback.
Numero di osservazioni (N)	50	10 combinazioni × 5 replicazioni.

- Importanza di `PageType`:

$$\frac{SS_{\text{PageType}}}{SST} = 99.57\% \quad (6)$$

- Importanza dell'Interazione:

$$\frac{SS_{\text{interazione}}}{SST} = 0.33\% \quad (7)$$

- Importanza dell'Errore:

$$\frac{SSE}{SST} = 0.10\% \quad (8)$$

- Importanza di `Intensity`:

$$\frac{SS_{\text{Intensity}}}{SST} = 0.00\% \quad (9)$$

5.4. Analisi di Significativitá e Verifica delle Ipotesi

L'applicabilità dell'ANOVA parametrica F-test richiede la verifica dei presupposti di normalità e omoschedasticità dei residui del modello ($\epsilon = y - \hat{y}$)

5.4.1. Test di Normalità sui Residui (Shapiro-Wilk)

Per verificare l'ipotesi nulla H_0 che i residui provengano da una distribuzione normale, è stato applicato il test di Shapiro-Wilk, idoneo per campioni con $N < 2000$:

- Statistica del Test: 0.682652
- p-value: 4.094195×10^{-9}

Poiché il p-value$\ll 0.05$, si rigetta l'ipotesi nulla H_0 di normalità. I residui presentano una forte deviazione dalla distribuzione normale.

5.4.2. Test Non Parametrico di Kruskal-Wallis

In presenza di residui non normali ed eteroschedastici (la varianza degli errori cresce all'aumentare della dimensione della pagina), i test parametrici tradizionali perdono di validità. Si ricorre quindi al test non parametrico ad una via di Kruskal-Wallis sui ranghi delle osservazioni:

1. Valutazione del fattore `Intensity`:
   - Statistica $\mathcal{X}^2$: 0.3619
   - p-value: 0.5474
   - Esito: Poiché $p = 0.5474 > 0.05$, non è possibile rigettare l'ipotesi nulla di uguaglianza delle distribuzioni. Il fattore `Intensity` NON è statisticamente significativo.
2. Valutazione del fattore `PageType`:
   - Statistica $\mathcal{X}^2$: 47.0792

Tabella 19. Analisi della varianza (ANOVA)

Origine della Varianza	DF	SS	MS	Rapporto F	Prob > F
`C(Intensity)`	1	0.020402	0.020402	0.670104	0.417865
`C(PageType)`	4	1188.328192	297.082048	9757.670893	3.357×10^{-59}
`C(Intensity):C(PageType)`	4	3.904048	0.976012	32.057150	5.419×10^{-12}
Residuo (Errore)	40	1.217840	0.030446	–	–
Totale Corretto (SST)	49	1193.470482	–	–	–

- p-value: 1.468×10^{-9}
- Esito: Poiché $p \ll 0.05$, si rigetta fortemente l'ipotesi nulla. Il fattore `PageType` è statisticamente significativo con un elevato livello di confidenza (> 99.9%)

5.5. Conclusioni

L'esperimento condotto ha consentito di caratterizzare con precisione il comportamento del Web Server e di validare empiricamente la teoria del Design of Experiments:

1. **Dominanza della Dimensione del File**: la dimensione della pagina erogata (`PageType`) determina in maniera pressoché esclusiva le prestazioni in termini di tempo di risposta (99.57% della varianza, $p < 10^8$).
2. **Ininfluenza dell'Intensità nel Range di Esercizio**: L'intensità del carico (`Intensity`), testata tra il 25% (375 req/min) e il 75% (1125 req/min) della capacità utile del server, non produce alcuna variazione statisticamente significativa nel tempo di risposta ($p = 0.5474$). Ciò dimostra che, entro i limiti della Usable Capacity, il server lavora in zona di stabilità senza fenomeni di saturazione o accodamento rilevanti.
3. **Rigore Metodologico**: La presenza di eteroschedasticità e la non-normalità degli errori hanno confermato la necessità di affiancare all'analisi ANOVA classica i test non parametrici (Kruskal-Wallis), giungendo a conclusioni statisticamente ineccepibili.

6. REGRESSION

6.1. Introduzione

Il presente progetto ha lo scopo di realizzare una piattaforma software in Python per il monitoraggio in tempo reale e l'analisi statistica avanzata del consumo di risorse di sistema (memoria fisica `RSS`, memoria virtuale `VmSize`, I/O).
L'obiettivo primario è identificare precocemente fenomeni di **Memory Leak** e stimare con accuratezza il **Tempo al Fallimento** ($T_{failure}$) prima che il sistema vada in saturazione (es. limite di 1GB).
I metodi di regressione lineare classica basati sui minimi quadrati (OLS) falliscono in presenza di:

- Assenza di normalità nei residui.
- Presenza di outlier o dinamiche non lineari.
- Invarianza o rumore pesante nelle misurazioni.

Per superare i limiti dell'OLS, il sistema implementa un **approccio non parametrico a due fasi**:

1. **Test di Mann-Kendall** per accertare la presenza di un trend monotono statisticamente significativo.
2. **Stimatore di Theil-Sen** per determinare in modo robusto la pendenza e l'intercetta della retta di regressione, con relativi intervalli di confidenza al 95%.

6.2. Pipeline metodologica statistica

Fase 1: Regressione Lineare Parametrica (OLS) Si calcola la retta $y = b + m \cdot x$ e l'indice di determinazione:

$$R^2 = 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y}_i)^2} \tag{10}$$

Un valore di $R^2 \approx 0$ o la presenza di pendenze incoerenti evidenziano l'inadeguatezza del modello lineare classico.

Fase 2: Test di Normalità dei Residui (KSL) I residui $e_i = y_i \hat{y}^i$ vengono standardizzati:

$$z_i = \frac{e_i - \bar{e}}{s_e} \tag{11}$$

Si applica il test di Kolmogorov-Smirnov. Se $p - \text{value} < 0.01$, si rifiuta l'ipotesi nulla H_0 (i dati non sono distribuiti normalmente), rendendo inapplicabile la regressione parametrica.

Fase 3: Test Non Parametrico di Trend (Mann-Kendall) Il test valuta la monotonia della serie temporale senza assumere alcuna distribuzione a priori:

$$\begin{aligned} S &= \sum_{k=1}^{n-1} \sum_{j=k+1}^{n} (y_j - y_k) \\ \tau &= \frac{S}{\frac{1}{2} n(n+1)} \end{aligned} \tag{12}$$

Un $p - \text{value} < 0.05$ conferma la presenza di un trend statisticamente significativo.

Fase 4: Stimatore Robusto di Theil-Sen Per ogni coppia di punti (x_i, y_i) e (x_j, y_j) con $x_i \neq x_j$, si calcola la pendenza:

$$S_{ij} = \frac{y_j - y_i}{x_j - x_i} \tag{13}$$

La pendenza stimata $\hat{m}$ è la mediana di tutte le pendenze S_{ij}.
L'intercetta è calcolata come $\hat{b} = mediana(y_i \hat{m} \cdot x_i)$. Se l'intervallo di confidenza al 95% $[m_{\text{inf}}, m_{\text{sup}}]$ non include lo 0, si afferma con il 95% di confidenza che la pendenza è significativamente diversa da zero.

Fase 5: Calcolo del Tempo al Fallimento ($T_{failure}$) Fissato un limite di saturazione C_{limit}:

$$T_{failure} = \frac{C_{\text{limit}-\hat{b}}}{\hat{m}} \tag{14}$$

L'incertezza ΔT derivante dal margine di errore dell'intervallo di confidenza al 95% è

$$\Delta T = \frac{1}{2} \left| \frac{C_{\text{limit}-\hat{b}}}{\hat{m}_{\text{inf}}} - \frac{C_{\text{limit}-\hat{b}}}{\hat{m}_{\text{sup}}} \right| \tag{15}$$

6.3. Sperimentazione e Risultati Empirici (Caso di Studio RSS)

- Durata Monitoraggio: 3.600 secondi (1 Ora).
- Intervallo di Campionamento: 1.0s.
- Metrica analizzata: RSS (*Resident Set Size* — RAM fisica occupata).
- Limite di Saturazione: 1GB.

Code 1. Report dei Risultati

Tabella 20. Confronto tra la regressione OLS e il metodo non parametrico Theil–Sen nell'analisi del trend di memoria.

Metrica / Test	Risultato OLS	Risultato non parametrico	Interpretazione
Pendenza (m)	$-9\,816{,}67$ B/s	$+155{,}48$ B/s	L'OLS restituisce una pendenza negativa, influenzata dalle fluttuazioni iniziali e dagli outlier. Theil–Sen evidenzia invece un trend crescente del consumo di memoria.
Normalità dei residui	$D = 0{,}0756$	$p = 3{,}14 \times 10^{-18}$	L'ipotesi nulla di normalità è ampiamente rifiutata ($p < 0{,}01$).
Significatività del trend	N.A.	$\tau = 0{,}3177$, $p = 1{,}80 \times 10^{-174}$	Il trend crescente è statisticamente significativo, con una correlazione positiva di intensità moderata.
Stima di T_{failure}	N.A. (pendenza < 0)	$0{,}21 \pm 0{,}01$ anni	Si stima il raggiungimento della soglia di saturazione di 1 GB in circa 75 giorni.

```
======================================================
REAL MEASUREMENT ANALYSIS: RSS vs TIME
======================================================
1. OLS LINEAR ESTIMATION:
- Equation: RSS = 76403580.252 - 9816.668397 * TIME
- R-squared (R): 0.444309
- RMSE: 11422241.606

2. RESIDUAL NORMALITY TEST (KSL / Kolmogorov-Smirnov):
- Statistic D: 0.075595
- p-value: 3.1444e-18
- Result: NOT Normal (H0 Hypothesis Rejected)

3. MANN-KENDALL TEST (TREND):
- Tau di Kendall (): 0.317701
- p-value: 1.8058e-174
- Presence of Trend: YES (Significance p < 0.05)

4. THEIL-SEN ESTIMATOR:
- Slope (m): 155.482945 Bytes/sec
- Intercept (b): 65276670.056 Bytes
- CI 95%: [145.362898, 163.650328]

5. FAILURE PREDICTION (Memory Saturation at 1 GB):
- Time to Failure: 6486018  387626 s
- Time to Failure (Years): 0.21  0.01 years (~75 giorni)
```

6.4. Conclusioni e Sviluppi Futuri

La sperimentazione ha dimostrato come la combinazione del Test di Mann-Kendall e dello Stimatore di Theil-Sen permetta di superare completamente le limitazioni della regressione lineare parametrica sui dati di monitoraggio di sistema. I vantaggi dimostrati sono:

- Insensibilità agli Outlier: Theil-Sen calcola la mediana delle pendenze, ignorando i picchi temporanei di memoria.
- Robustezza Matematica: La non-normalità dei residui non compromette la validità della stima.
- Predizione Affidabile: Il calcolo del tempo di fallimento fornisce intervalli di confidenza rigorosi utili per la manutenzione preventiva (Aging Rejuvenation).

7. BENCHMARK

I

8. FFDA

I

9. RDB E FAULT TREE

I

RIFERIMENTI BIBLIOGRAFICI

[1] T. Zhang, R. Ramakrishnan e M. Livny, «BIRCH: an efficient data clustering method for very large databases,» *SIGMOD Rec.*, vol. 25, n. 2, pp. 103–114, giu. 1996, ISSN: 0163-5808. DOI: 10.1145/235968.233324 indirizzo: https://doi.org/10.1145/235968.233324